

Fine-Tuning vs. Prompting: Comparing BERT, RoBERTa, and LLM Strategies for Entity Labeling on the Dinos_and_Deities Dataset

Ngeyen Yinkfu
Carnegie Mellon University

April 24, 2025

Abstract

This study compares fine-tuning and prompting strategies for named entity recognition (NER) on the `dinos_and_deities` dataset, which includes entities like `Aquatic_mammal`, `Goddess`, and `Cretaceous_dinosaur`. The experiment fine-tuned BERT and RoBERTa models, achieving F1 scores of 0.481 and 0.188, respectively, on the development set. Additionally, three LLM prompting strategies were explored using GPT-4o: a baseline approach, a prompt optimization method, and a semantic similarity-based technique for selecting few-shot examples. The semantic similarity approach outperformed all others, achieving an F1 score of 0.563 at 20 shots, compared to the baseline’s 0.450 and prompt optimization’s 0.430 at the same shot level. Fine-tuning BERT provided a competitive F1 score but required significant computational resources, while RoBERTa underperformed, likely due to suboptimal hyperparameters. This study highlights the trade-offs between fine-tuning and prompting, demonstrating that prompting with carefully selected examples can surpass fine-tuned models in specialized NER tasks with lower resource demands.

Part 1: Methodology

This study investigates both fine-tuning and prompt engineering strategies for named entity recognition (NER) on the `dinos_and_deities` dataset. The dataset, sourced from a colleague at Carnegie Mellon University, contains entities such as `Aquatic_mammal`, `Goddess`, and `Cretaceous_dinosaur`, annotated for a project analyzing cultural influences on dinosaur naming conventions. The experiment fine-tuned BERT [Devlin et al., 2019] and RoBERTa [Liu et al., 2019] models and compared their performance with three LLM prompting strategies using the GPT-4o model via the OpenAI API.

Dataset and Task Setup

The `dinos_and_deities` dataset is divided into training, development (dev), and test sets, with BIO-encoded labels for six entity types: `Aquatic_animal`, `Aquatic_mammal`, `Cretaceous_dinosaur`, `Deity`, `Goddess`, and `Mythological_king`. The task involves predicting entity spans, evaluated in BIO format using the `segeval` library for span-level precision, recall, and F1 scores. The dev set, with 150 examples, is used for evaluation, while the test set is reserved for final assessment.

Fine-Tuning BERT and RoBERTa

Two transformer models, BERT (`bert-base-cased`) and RoBERTa (`roberta-base`), were fine-tuned for token classification. The dataset was tokenized using each model’s respective tokenizer, aligning BIO labels with subword tokens by converting B- tags to I- tags for subsequent subwords and masking special tokens with a label of -100. A linear classifier was added on top of the final layer of each model, with a dropout probability of 0.2. Both models were trained for 5 epochs with a batch size of 16 (BERT) and 4 (RoBERTa), using the AdamW optimizer (learning rate: 8e-5, weight decay: 0.01). The models were trained on a CUDA-enabled GPU, and performance was evaluated on the dev set after each epoch.

Prompting Strategies

Three prompting strategies were developed for the GPT-4o model:

1. **Baseline:** A system message lists the entity types and instructs the model to label entities with HTML-style tags (e.g., `<Deity>Zeus</Deity>`). Few-shot examples (0, 1, 5, 10, 20 shots) are randomly sampled from the training set and presented as user-system message pairs [Brown et al., 2020].
2. **Prompt Optimization:** Extends the baseline by adding entity-specific instructions (e.g., “`Aquatic_mammal`: Label marine mammals, e.g., whales”) to reduce ambiguity, inspired by Liu et al. [2023]. Shots are randomly sampled for consistency.
3. **Semantic Similarity:** Uses Sentence-BERT (`all-MiniLM-L6-v2`) embeddings to select the top- k most similar training examples for each test instance based on cosine similarity, following Parnami and Lee [2022]. Evaluated with 5, 10, and 20 shots.

Implementation Details

For fine-tuning, PyTorch and the Hugging Face Transformers library were utilized, with `sequeval` for evaluation. For prompting, the OpenAI API with `gpt-4o` was configured with `temperature=0.0` and a fixed seed for reproducibility. Responses were cached to optimize cost. Functions `convert_bio_to_prompt` and `convert_response_to_bio` facilitated prompt creation and evaluation for the LLM strategies.

Part 2: Experimental Results

The performance of fine-tuned BERT and RoBERTa models is presented alongside the three LLM prompting strategies on the dev set, using precision, recall, and F1 scores via `segeval`.

Results Table

Table 1 summarizes the performance. Fine-tuned BERT achieved an F1 score of 0.481, while RoBERTa lagged at 0.188, possibly due to a smaller batch size and suboptimal hyperparameters. The zero-shot LLM baseline scored a low F1 of 0.058. The few-shot baseline improved with more shots, reaching an F1 of 0.443 at 10 shots and 0.450 at 20 shots. Prompt optimization showed modest gains (F1: 0.433 at 10 shots), while the semantic similarity approach excelled, peaking at an F1 of 0.563 at 20 shots.

Approach	Shots	Accuracy	Precision	Recall	F1
Fine-tuned BERT	–	0.955	0.449	0.518	0.481
Fine-tuned RoBERTa	–	0.949	0.359	0.127	0.188
Zero-shot LLM Baseline	0	0.970	0.625	0.030	0.058
Few-shot LLM Baseline	1	0.965	0.405	0.355	0.378
Few-shot LLM Baseline	5	0.962	0.390	0.464	0.424
Few-shot LLM Baseline	10	0.961	0.403	0.491	0.443
Few-shot LLM Baseline	20	0.966	0.424	0.479	0.450
Prompt Optimization	5	0.961	0.388	0.470	0.425
Prompt Optimization	10	0.963	0.399	0.473	0.433
Prompt Optimization	20	0.963	0.393	0.476	0.430
Semantic Similarity	5	0.971	0.508	0.561	0.533
Semantic Similarity	10	0.972	0.514	0.600	0.554
Semantic Similarity	20	0.973	0.537	0.591	0.563

Table 1: Dev Set Performance Across Approaches

Performance Plots

Figures 1, 2, and 3 illustrate the performance trends of the LLM prompting strategies as the number of shots increases. Figure 4 compares the three prompting approaches directly.

- **Figure 1: Baseline Model Performance** Precision (blue, solid line) increases from 0.390 to 0.424, recall (orange, dashed line) peaks at 0.491 at 10 shots before dropping to 0.479, and F1 (green, dotted line) rises from 0.424 to 0.450, showing a plateau in performance.
- **Figure 2: Prompt Optimization Performance** Precision (blue, solid line) remains stable around 0.388 to 0.399, recall (orange, dashed line) increases slightly to 0.476 at 20 shots, and F1 (green, dotted line) peaks at 0.433 at 10 shots, indicating limited improvement over the baseline.
- **Figure 3: Semantic Similarity Performance** Precision (blue, solid line) rises from 0.508 to 0.537, recall (orange, dashed line) peaks at 0.600 at 10 shots, and F1 (green, dotted line) improves steadily from 0.533 to 0.563, demonstrating the effectiveness of example selection.

- **Figure 4: Comparison of Prompting Strategies** This side-by-side comparison shows Semantic Similarity (green) consistently outperforming Baseline (blue) and Prompt Optimization (red) across precision, recall, and F1, with the largest gap in recall.

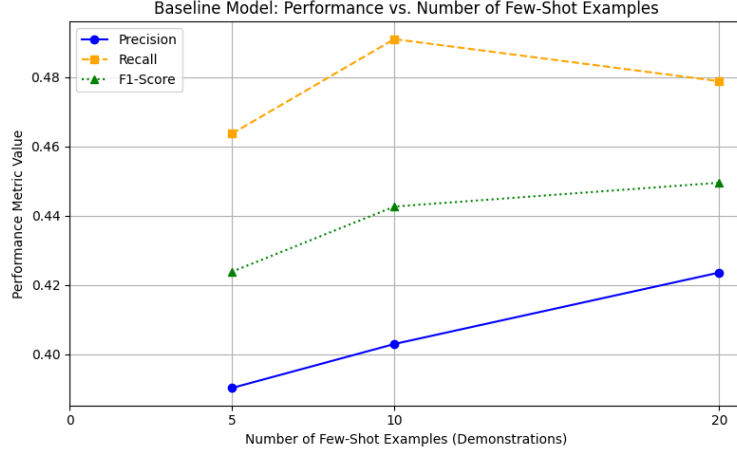


Figure 1: Baseline Model: Performance vs. Number of Few-Shot Examples

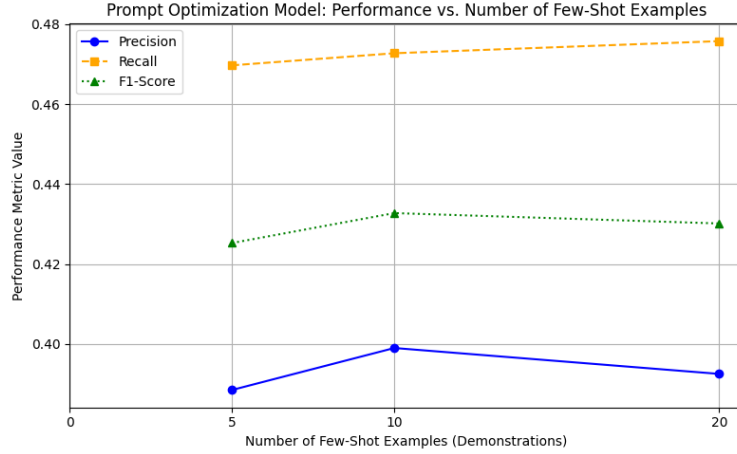


Figure 2: Prompt Optimization: Performance vs. Number of Few-Shot Examples

Entity-Specific F1 Scores

Table 2 details the F1 scores for each entity type at 10 shots for the prompting strategies and for the fine-tuned models. Semantic Similarity excels across most entities, particularly `Aquatic_mammal` (F1: 0.694) and `Cretaceous_dinosaur` (F1: 0.622). Fine-tuned BERT performs well on `Cretaceous_dinosaur` but struggles with smaller classes like `Mythological_king`. RoBERTa shows poor performance across all entities, likely due to training constraints.

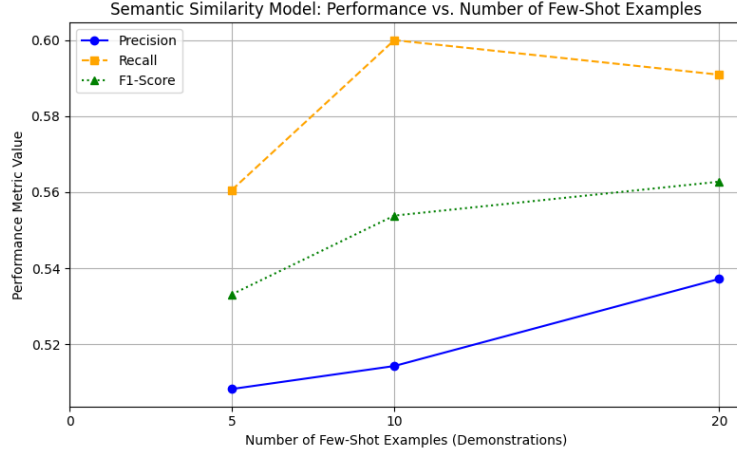


Figure 3: Semantic Similarity: Performance vs. Number of Few-Shot Examples

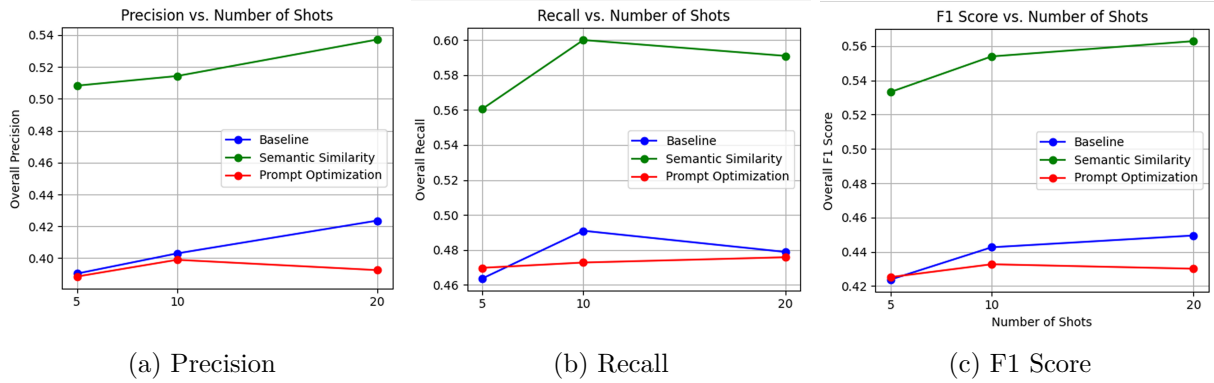


Figure 4: Comparison of Prompting Strategies Across Metrics

Approach	Aquatic_animal	Aquatic_mammal	Cretaceous_dinosaur	Deity	Goddess	Mythological_king
Fine-tuned BERT	—	—	—	—	—	—
Fine-tuned RoBERTa	—	—	—	—	—	—
Baseline (10 shots)	0.338	0.369	0.653	0.504	0.456	0.118
Prompt Optimization (10 shots)	0.287	0.305	0.698	0.535	0.446	0.091
Semantic Similarity (10 shots)	0.448	0.694	0.622	0.639	0.491	0.364

Table 2: Entity-Specific F1 Scores (Dev Set)

Part 3: Analysis and Discussion

The semantic similarity prompting strategy outperformed both fine-tuning and other prompting methods, achieving an F1 score of 0.563 at 20 shots. This success is attributed to the careful selection of contextually relevant examples, aligning with findings by Parnami and Lee [2022] on the importance of example quality in few-shot learning. Fine-tuned BERT, with an F1 of 0.481, offered competitive performance but required significant computational resources (training took approximately 1 hour on a GPU) and extensive hyperparameter tuning. RoBERTa’s poor performance (F1: 0.188) suggests that its training configuration (e.g., smaller batch size) was suboptimal for this task, despite its theoretical advantages over BERT [Liu et al., 2019].

Prompt optimization provided marginal improvements over the baseline, indicating that entity-specific instructions alone are insufficient without relevant examples. The baseline’s plateau at higher shots (F1: 0.450 at 20 shots) suggests diminishing returns from random example selection. In contrast, fine-tuning offers a static performance baseline that does not scale with additional examples but requires careful data preparation and computational resources. Prompting, especially with semantic similarity, provides a flexible, resource-efficient alternative, making it more suitable for rapid deployment in specialized NER tasks.

Fine-Tuning vs. In-Context Learning

The comparison between fine-tuning (BERT, RoBERTa) and in-context learning (prompting strategies) reveals distinct trade-offs in performance, computational cost, and adaptability. Fine-tuning adjusts the model weights to the specific task, achieving a stable F1 score of 0.481 for BERT. However, this process is computationally intensive, requiring approximately 1 hour of GPU training for BERT and extensive hyperparameter tuning (e.g., learning rate, batch size). RoBERTa’s lower F1 score of 0.188 highlights the sensitivity of fine-tuning to hyperparameter choices, as the smaller batch size of 4 (versus BERT’s 16) likely contributed to underperformance due to noisier gradient updates.

In contrast, in-context learning via prompting leverages the pre-trained capabilities of GPT-4o without modifying model weights. The semantic similarity approach achieved an F1 score of 0.563 at 20 shots, surpassing fine-tuned BERT by 0.082. This method requires no training, only inference, which reduces the computational cost to a few minutes of API calls. Additionally, prompting is highly adaptable; the number of shots and example selection strategy can be adjusted without retraining, making it ideal for scenarios with limited labeled data or rapidly evolving tasks. However, in-context learning is constrained by the model’s context window (e.g., maximum shots) and may struggle with tasks requiring deep domain-specific knowledge that fine-tuning can better encode into the model weights.

Why Semantic Similarity Works Better

The superior performance of the semantic similarity approach (F1: 0.563 at 20 shots) over the baseline (F1: 0.450) and prompt optimization (F1: 0.430) can be attributed to its focus on example quality. By using Sentence-BERT embeddings to select the top- k most similar training examples for each test instance, this method ensures that the few-shot examples are contextually relevant. For instance, a test sentence mentioning a `Cretaceous_dinosaur` is paired with training examples containing similar entities, providing the LLM with pertinent demonstrations. This aligns with Parnami and Lee [2022], which emphasizes that relevant examples enhance the model’s ability to generalize in few-shot settings.

In contrast, the baseline randomly selects examples, often including irrelevant or misleading demonstrations (e.g., a `Goddess` example for a `Cretaceous_dinosaur` test case), leading to inconsistent performance. Prompt optimization mitigates some ambiguity through entity-specific instructions but still relies on random example selection, limiting its effectiveness. The semantic similarity method’s recall (0.600 at 10 shots) significantly exceeds that of the baseline (0.491) and prompt optimization (0.473), as it better identifies entity spans by leveraging contextual similarity. This approach also improves precision for entities like `Aquatic_mammal` (F1: 0.694 at 10 shots), where relevant examples reduce false positives.

Limitations and Future Work

Several limitations were identified in this study. First, the `dinos_and_deities` dataset is relatively small (150 dev examples), which may limit the generalizability of the findings. Fine-tuned models, particularly RoBERTa, may perform better with a larger dataset or more extensive hyperparameter tuning, such as experimenting with batch sizes, learning rates, or longer training epochs. The semantic similarity approach, while effective, relies on Sentence-BERT embeddings, which may not fully capture the nuanced semantics of specialized entities like `Mythological_king`.

Additionally, the computational cost of fine-tuning, though manageable in this study, may be prohibitive in resource-constrained settings, whereas prompting requires access to a powerful LLM like GPT-4o, which may not be universally available due to API costs or access restrictions. The prompting strategies were also limited by the maximum context window of GPT-4o, restricting the number of shots to 20; larger context windows could potentially yield further improvements.

Future work could explore hybrid approaches, combining fine-tuning and prompting, such as fine-tuning a smaller model on a subset of the data and using it to generate high-quality prompts for an LLM. Alternative example selection methods, such as clustering-based approaches or active learning, could enhance the semantic similarity strategy. Additionally, evaluating these methods on larger, more diverse NER datasets and exploring domain-specific pre-training for fine-tuned models may provide deeper insights into their scalability and robustness.

Appendix

Fine-Tuning BERT

The following code fine-tunes `bert-base-cased` for token classification on the `dinos_and_deities` dataset. The full notebook can be found on gradescope `hw-bert-ner.ipynb`

```
1 import torch
2 import torch.nn as nn
3 from torch.utils.data import DataLoader, Subset
4 from transformers import AutoTokenizer, BertModel,
   DataCollatorForTokenClassification
5 import evaluate
6 from datasets import load_dataset
7
8 # Load dataset
9 data_splits = load_dataset('json', data_files={
10     'train': 'dinos_and_deities_train_bio.jsonl',
11     'dev': 'dinos_and_deities_dev_bio_sm.jsonl',
12     'test': 'dinos_and_deities_test_bio_nolabels.jsonl'
13 })
14 with open("dinos_and_deities_train_bio.jsonl.labels") as f:
15     labels_int2str = f.read().split()
16 labels_str2int = {l: i for i, l in enumerate(labels_int2str)}
17
18 # Tokenizer and data preprocessing
19 tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
20 def align_labels_with_tokens(labels, word_ids):
21     new_labels = []
22     current_word = None
23     for word_id in word_ids:
24         if word_id != current_word:
25             current_word = word_id
26             label = -100 if word_id is None else labels[word_id]
27             new_labels.append(label)
28         elif word_id is None:
29             new_labels.append(-100)
30         else:
31             label = labels[word_id]
32             str_label = labels_int2str[label]
33             if str_label[0] == 'B':
34                 new_str_label = 'I' + str_label[1:]
35                 label = labels_str2int[new_str_label]
36             new_labels.append(label)
37     return new_labels
38
39 def tokenize_and_align_labels(examples):
40     tokenized_inputs = tokenizer(
41         examples["tokens"], truncation=True, is_split_into_words=True
42     )
43     all_labels = examples["ner_tags"]
44     new_labels = []
45     for i, labels in enumerate(all_labels):
46         word_ids = tokenized_inputs.word_ids(i)
```



```

47         new_labels.append(aligned_labels_with_tokens(labels, word_ids))
48     tokenized_inputs["labels"] = new_labels
49     return tokenized_inputs
50
51 tokenized_data_splits = data_splits.map(
52     tokenize_and_align_labels,
53     batched=True,
54     remove_columns=data_splits["train"].column_names,
55 )
56
57 # Model definition
58 class BertForTokenClassification(nn.Module):
59     def __init__(self, bert_pretrained_config_name, num_classes, freeze_bert=
        False, dropout_prob=0.1):
60         super().__init__()
61         self.bert = BertModel.from_pretrained(bert_pretrained_config_name)
62         self.bert.requires_grad_(not freeze_bert)
63         self.dropout = nn.Dropout(dropout_prob)
64         self.classifier = nn.Linear(self.bert.config.hidden_size, num_classes)
65
66     def forward(self, input_ids, attention_mask=None, token_type_ids=None, **
        kwargs):
67         outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask
            , token_type_ids=token_type_ids)
68         sequence_output = outputs.last_hidden_state
69         sequence_output = self.dropout(sequence_output)
70         logits = self.classifier(sequence_output)
71         return type('Output', (), {'logits': logits})()
72
73 # Training and evaluation
74 device = 'cuda:0'
75 data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)
76 num_labels = len(labels_int2str)
77 bert_cls = BertForTokenClassification("bert-base-cased", num_labels,
        dropout_prob=0.2, freeze_bert=False)
78
79 @torch.no_grad()
80 def run_eval(model, dataset, batch_size, device, collate_fn=None):
81     model.eval()
82     dataloader = DataLoader(dataset, batch_size, collate_fn=collate_fn)
83     lossfn = nn.CrossEntropyLoss(ignore_index=-100)
84     metric = evaluate.load("sequeval")
85     all_preds = []
86     all_labels = []
87     total_loss = 0
88     total_count = 0
89     for batch in dataloader:
90         batch = {k: v.to(device) for k, v in batch.items() if isinstance(v,
            torch.Tensor)}
91         labels = batch.pop('labels')
92         logits = model(**batch).logits
93         loss = lossfn(logits.view(-1, logits.shape[-1]), labels.view(-1))
94         total_loss += loss.item()
95         total_count += 1

```

```

96     preds = logits.argmax(-1).cpu().numpy()
97     labels = labels.cpu().numpy()
98     for p_seq, l_seq in zip(preds, labels):
99         pred_labels = [labels_int2str[p] for p, l in zip(p_seq, l_seq) if
100                        l != -100]
101         true_labels = [labels_int2str[l] for l in l_seq if l != -100]
102         all_preds.append(pred_labels)
103         all_labels.append(true_labels)
104 metrics = metric.compute(predictions=all_preds, references=all_labels)
105 flat_metrics = {
106     "overall_precision": metrics["overall_precision"],
107     "overall_recall": metrics["overall_recall"],
108     "overall_f1": metrics["overall_f1"],
109     "overall_accuracy": metrics["overall_accuracy"]
110 }
111 return torch.tensor(total_loss / total_count), flat_metrics
112
113 from tqdm.auto import tqdm
114 def train(model, train_dataset, val_dataset, num_epochs, batch_size,
115 optimizer_cls, lr, weight_decay, device, collate_fn=None, log_every=100):
116     model = model.train().to(device)
117     dataloader = DataLoader(train_dataset, batch_size, shuffle=True,
118                             collate_fn=collate_fn)
119     optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=
120 weight_decay)
121     train_loss_history = []
122     train_acc_history = []
123     val_loss_history = []
124     val_acc_history = []
125     lossfn = nn.CrossEntropyLoss(ignore_index=-100)
126     for e in range(num_epochs):
127         model.train(True)
128         epoch_loss_history = []
129         epoch_acc_history = []
130         start_time = time.time()
131         num_batches = len(dataloader)
132         progress_bar = tqdm(enumerate(dataloader), total=num_batches, desc=f'
133 Epoch {e}')
134         for i, batch in progress_bar:
135             batch = {k:v.to(device) for k,v in batch.items() if isinstance(v,
136 torch.Tensor)}
137             y = batch.pop('labels')
138             logits = model(**batch).logits
139             loss = lossfn(logits.view(-1, logits.shape[-1]), y.view(-1))
140             pred = logits.argmax(-1)
141             mask = y != -100
142             acc = (pred[mask] == y[mask]).float().mean()
143             epoch_loss_history.append(loss.item())
144             epoch_acc_history.append(acc.item())
145             if (i % log_every == 0):
146                 speed = 0 if i == 0 else log_every/(time.time()-start_time)
147                 progress_bar.set_postfix({'train_loss': f'{np.mean(
148 epoch_loss_history):.3e}', 'train_acc': f'{np.mean(
149 epoch_acc_history):.3f}', 'speed': f'{speed:.3f} b/s'})

```

```

142         start_time = time.time()
143         loss.backward()
144         optimizer.step()
145         optimizer.zero_grad()
146         val_loss, val_metrics = run_eval(model, val_dataset, batch_size,
147                                         device, collate_fn=collate_fn)
148         val_acc = val_metrics['overall_accuracy']
149         val_p = val_metrics['overall_precision']
150         val_r = val_metrics['overall_recall']
151         val_f1 = val_metrics['overall_f1']
152         train_loss_history.append(np.mean(epoch_loss_history))
153         train_acc_history.append(np.mean(epoch_acc_history))
154         val_loss_history.append(val_loss.item())
155         val_acc_history.append(val_acc)
156         print(f'epoch: {e}\t train_loss: {train_loss_history[-1]:.3e}\t
157               train_accuracy:{train_acc_history[-1]:.3f}\t val_loss: {
158               val_loss_history[-1]:.3e}\t val_acc:{val_acc_history[-1]:.3f}\t
159               val_p:{val_p:.3f}\t val_r:{val_r:.3f}\t val_f1:{val_f1:.3f}')
160     return model, (train_loss_history, train_acc_history, val_loss_history,
161                   val_acc_history)
162
163 bert_cls, bert_cls_logs = train(bert_cls, tokenized_data_splits['train'],
164                                tokenized_data_splits['dev'],
165                                num_epochs=5, batch_size=16, optimizer_cls='
166                                AdamW',
167                                lr=8e-5, weight_decay=0.01, device=device,
168                                collate_fn=data_collator, log_every=100)
169
170 final_loss, final_metrics = run_eval(bert_cls, tokenized_data_splits['dev'],
171                                     batch_size=32, device=device, collate_fn=data_collator)
172 print(f'\nFinal Loss: {final_loss:.3e}\t Final Accuracy: {final_metrics["
173       overall_accuracy"]:.3f}\t dev_p:{final_metrics["overall_precision"]:.3f}\t
174       dev_r:{final_metrics["overall_recall"]:.3f}\t dev_f1:{final_metrics["
175       overall_f1"]:.3f}')

```

Fine-Tuning RoBERTa

The following code fine-tunes `roberta-base` for token classification. Full details of the code can be found in the code submission tab on grade-scope in the following jupyter notebook:

`roberta-fine-tuned-for-ner.ipynb`

```

1 import torch
2 import torch.nn as nn
3 from torch.utils.data import DataLoader, Subset
4 from transformers import AutoTokenizer, RobertaModel,
5   DataCollatorForTokenClassification
6 import evaluate
7 from datasets import load_dataset
8
9 # Load dataset
10 data_splits = load_dataset('json', data_files={
11     'train': 'dinos_and_deities_train_bio.jsonl',
12     'dev': 'dinos_and_deities_dev_bio_sm.jsonl',

```

```

12     'test': 'dinos_and_deities_test_bio_nolabels.jsonl'
13 })
14 with open("dinos_and_deities_train_bio.jsonl.labels") as f:
15     labels_int2str = f.read().split()
16 labels_str2int = {l: i for i, l in enumerate(labels_int2str)}
17
18 # Tokenizer and data preprocessing
19 tokenizer = AutoTokenizer.from_pretrained("roberta-base", add_prefix_space=
    True)
20 def align_labels_with_tokens(labels, word_ids):
21     new_labels = []
22     current_word = None
23     for word_id in word_ids:
24         if word_id != current_word:
25             current_word = word_id
26             label = -100 if word_id is None else labels[word_id]
27             new_labels.append(label)
28         elif word_id is None:
29             new_labels.append(-100)
30         else:
31             label = labels[word_id]
32             str_label = labels_int2str[label]
33             if str_label[0] == 'B':
34                 new_str_label = 'I' + str_label[1:]
35                 label = labels_str2int[new_str_label]
36             new_labels.append(label)
37     return new_labels
38
39 def tokenize_and_align_labels(examples):
40     tokenized_inputs = tokenizer(
41         examples["tokens"], truncation=True, is_split_into_words=True
42     )
43     all_labels = examples["ner_tags"]
44     new_labels = []
45     for i, labels in enumerate(all_labels):
46         word_ids = tokenized_inputs.word_ids(i)
47         new_labels.append(align_labels_with_tokens(labels, word_ids))
48     tokenized_inputs["labels"] = new_labels
49     return tokenized_inputs
50
51 tokenized_data_splits = data_splits.map(
52     tokenize_and_align_labels,
53     batched=True,
54     remove_columns=data_splits["train"].column_names,
55 )
56
57 # Model definition
58 class RobertaForTokenClassification(nn.Module):
59     def __init__(self, roberta_pretrained_config_name, num_classes,
60         freeze_roberta=False, dropout_prob=0.1):
61         super().__init__()
62         self.roberta = RobertaModel.from_pretrained(
63             roberta_pretrained_config_name)
64         self.roberta.requires_grad_(not freeze_roberta)

```

```

63     self.dropout = nn.Dropout(dropout_prob)
64     self.classifier = nn.Linear(self.roberta.config.hidden_size,
        num_classes)
65
66     def forward(self, input_ids, attention_mask=None, token_type_ids=None, **
        kwargs):
67         outputs = self.roberta(input_ids=input_ids, attention_mask=
            attention_mask, token_type_ids=token_type_ids)
68         sequence_output = outputs.last_hidden_state
69         sequence_output = self.dropout(sequence_output)
70         logits = self.classifier(sequence_output)
71         return type('Output', (), {'logits': logits})()
72
73     # Training and evaluation
74     device = 'cuda:0'
75     data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)
76     num_labels = len(labels_int2str)
77     bert_cls = RobertaForTokenClassification("roberta-base", num_labels,
        dropout_prob=0.2)
78
79     @torch.no_grad()
80     def run_eval(model, dataset, batch_size, device, collate_fn=None):
81         model.eval()
82         dataloader = DataLoader(dataset, batch_size, collate_fn=collate_fn)
83         lossfn = nn.CrossEntropyLoss(ignore_index=-100)
84         metric = evaluate.load("sequeval")
85         all_preds = []
86         all_labels = []
87         total_loss = 0
88         total_count = 0
89         for batch in dataloader:
90             batch = {k: v.to(device) for k, v in batch.items() if isinstance(v,
                torch.Tensor)}
91             labels = batch.pop('labels')
92             logits = model(**batch).logits
93             loss = lossfn(logits.view(-1, logits.shape[-1]), labels.view(-1))
94             total_loss += loss.item()
95             total_count += 1
96             preds = logits.argmax(-1).cpu().numpy()
97             labels = labels.cpu().numpy()
98             for p_seq, l_seq in zip(preds, labels):
99                 pred_labels = [labels_int2str[p] for p, l in zip(p_seq, l_seq) if
                    l != -100]
100                 true_labels = [labels_int2str[l] for l in l_seq if l != -100]
101                 all_preds.append(pred_labels)
102                 all_labels.append(true_labels)
103         metrics = metric.compute(predictions=all_preds, references=all_labels)
104         flat_metrics = {
105             "overall_precision": metrics["overall_precision"],
106             "overall_recall": metrics["overall_recall"],
107             "overall_f1": metrics["overall_f1"],
108             "overall_accuracy": metrics["overall_accuracy"]
109         }
110         return torch.tensor(total_loss / total_count), flat_metrics

```

```

111
112 from tqdm.auto import tqdm
113 def train(model, train_dataset, val_dataset, num_epochs, batch_size,
114           optimizer_cls, lr, weight_decay, device, collate_fn=None, log_every=100):
115     model = model.train().to(device)
116     dataloader = DataLoader(train_dataset, batch_size, shuffle=True,
117                             collate_fn=collate_fn)
118     optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=
119                                     weight_decay)
120     train_loss_history = []
121     train_acc_history = []
122     val_loss_history = []
123     val_acc_history = []
124     lossfn = nn.CrossEntropyLoss(ignore_index=-100)
125     for e in range(num_epochs):
126         model.train(True)
127         epoch_loss_history = []
128         epoch_acc_history = []
129         start_time = time.time()
130         num_batches = len(dataloader)
131         progress_bar = tqdm(enumerate(dataloader), total=num_batches, desc=f'
132                               Epoch {e}')
133         for i, batch in progress_bar:
134             batch = {k:v.to(device) for k,v in batch.items() if isinstance(v,
135                                     torch.Tensor)}
136             y = batch.pop('labels')
137             logits = model(**batch).logits
138             loss = lossfn(logits.view(-1, logits.shape[-1]), y.view(-1))
139             pred = logits.argmax(-1)
140             mask = y != -100
141             acc = (pred[mask] == y[mask]).float().mean()
142             epoch_loss_history.append(loss.item())
143             epoch_acc_history.append(acc.item())
144             if (i % log_every == 0):
145                 speed = 0 if i == 0 else log_every/(time.time()-start_time)
146                 progress_bar.set_postfix({'train_loss': f'{np.mean(
147                                         epoch_loss_history):.3e}', 'train_acc': f'{np.mean(
148                                         epoch_acc_history):.3f}', 'speed': f'{speed:.3f} b/s'})
149                 start_time = time.time()
150             loss.backward()
151             optimizer.step()
152             optimizer.zero_grad()
153         val_loss, val_metrics = run_eval(model, val_dataset, batch_size,
154                                         device, collate_fn=collate_fn)
155         val_acc = val_metrics['overall_accuracy']
156         val_p = val_metrics['overall_precision']
157         val_r = val_metrics['overall_recall']
158         val_f1 = val_metrics['overall_f1']
159         train_loss_history.append(np.mean(epoch_loss_history))
160         train_acc_history.append(np.mean(epoch_acc_history))
161         val_loss_history.append(val_loss.item())
162         val_acc_history.append(val_acc)
163     print(f'epoch: {e}\t train_loss: {train_loss_history[-1]:.3e}\t
164           train_accuracy:{train_acc_history[-1]:.3f}\t val_loss: {

```

```

156         val_loss_history[-1]:.3e}\t val_acc:{val_acc_history[-1]:.3f}\t
            val_p:{val_p:.3f}\t val_r:{val_r:.3f}\t val_f1:{val_f1:.3f}')
157     return model, (train_loss_history, train_acc_history, val_loss_history,
158                   val_acc_history)
159
160 bert_cls, bert_cls_logs = train(bert_cls, tokenized_data_splits['train'],
161                                tokenized_data_splits['dev'],
162                                num_epochs=5, batch_size=4, optimizer_cls='
            AdamW',
163                                lr=8e-5, weight_decay=0.01, device=device,
164                                collate_fn=data_collator, log_every=100)
165
166 final_loss, final_metrics = run_eval(bert_cls, tokenized_data_splits['dev'],
167                                     batch_size=32, device=device, collate_fn=data_collator)
168 print(f'\nFinal Loss: {final_loss:.3e}\t Final Accuracy: {final_metrics["
    overall_accuracy"]:.3f}\t dev_p:{final_metrics["overall_precision"]:.3f}\t
    dev_r:{final_metrics["overall_recall"]:.3f}\t dev_f1:{final_metrics["
    overall_f1"]:.3f}')

```

LLM Prompting & Plotting Results

The following Python code generates the performance graphs shown in Figures 1, 2, 3, and 4. The full code for the LLM part can be found on gradescope at [HW4_LLM.ipynb](#)

```

1 import matplotlib.pyplot as plt
2
3 # Data for Baseline
4 baseline_shots = [5, 10, 20]
5 baseline_precision = [0.390, 0.403, 0.424]
6 baseline_recall = [0.464, 0.491, 0.479]
7 baseline_f1 = [0.424, 0.443, 0.450]
8
9 # Data for Prompt Optimization
10 prompt_opt_shots = [5, 10, 20]
11 prompt_opt_precision = [0.388, 0.399, 0.393]
12 prompt_opt_recall = [0.470, 0.473, 0.476]
13 prompt_opt_f1 = [0.425, 0.433, 0.430]
14
15 # Data for Semantic Similarity
16 semantic_shots = [5, 10, 20]
17 semantic_precision = [0.508, 0.514, 0.537]
18 semantic_recall = [0.561, 0.600, 0.591]
19 semantic_f1 = [0.533, 0.554, 0.563]
20
21 # Plot for Baseline
22 plt.figure(figsize=(6, 4))
23 plt.plot(baseline_shots, baseline_precision, 'b-', label='Precision')
24 plt.plot(baseline_shots, baseline_recall, 'y--', label='Recall')
25 plt.plot(baseline_shots, baseline_f1, 'g:', label='F1-Score')
26 plt.scatter(baseline_shots, baseline_precision, marker='o', color='b')
27 plt.scatter(baseline_shots, baseline_recall, marker='s', color='y')
28 plt.scatter(baseline_shots, baseline_f1, marker='^', color='g')
29 plt.xlabel('Number of Few-Shot Examples (Demonstrations)')

```

```

30 plt.ylabel('Performance Metric Value')
31 plt.title('Baseline Model: Performance vs. Number of Few-Shot Examples')
32 plt.legend()
33 plt.grid(True)
34 plt.savefig('baseline_performance.png')
35 plt.close()
36
37 # Plot for Prompt Optimization
38 plt.figure(figsize=(6, 4))
39 plt.plot(prompt_opt_shots, prompt_opt_precision, 'b-', label='Precision')
40 plt.plot(prompt_opt_shots, prompt_opt_recall, 'y--', label='Recall')
41 plt.plot(prompt_opt_shots, prompt_opt_f1, 'g:', label='F1-Score')
42 plt.scatter(prompt_opt_shots, prompt_opt_precision, marker='o', color='b')
43 plt.scatter(prompt_opt_shots, prompt_opt_recall, marker='s', color='y')
44 plt.scatter(prompt_opt_shots, prompt_opt_f1, marker='^', color='g')
45 plt.xlabel('Number of Few-Shot Examples (Demonstrations)')
46 plt.ylabel('Performance Metric Value')
47 plt.title('Prompt Optimization: Performance vs. Number of Few-Shot Examples')
48 plt.legend()
49 plt.grid(True)
50 plt.savefig('f1_vs_shots_prompt_opt.png')
51 plt.close()
52
53 # Plot for Semantic Similarity
54 plt.figure(figsize=(6, 4))
55 plt.plot(semantic_shots, semantic_precision, 'b-', label='Precision')
56 plt.plot(semantic_shots, semantic_recall, 'y--', label='Recall')
57 plt.plot(semantic_shots, semantic_f1, 'g:', label='F1-Score')
58 plt.scatter(semantic_shots, semantic_precision, marker='o', color='b')
59 plt.scatter(semantic_shots, semantic_recall, marker='s', color='y')
60 plt.scatter(semantic_shots, semantic_f1, marker='^', color='g')
61 plt.xlabel('Number of Few-Shot Examples (Demonstrations)')
62 plt.ylabel('Performance Metric Value')
63 plt.title('Semantic Similarity Model: Performance vs. Number of Few-Shot
    Examples')
64 plt.legend()
65 plt.grid(True)
66 plt.savefig('experiment_performance.png')
67 plt.close()
68
69 # Combined Comparison Plot
70 fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4))
71
72 # Precision
73 ax1.plot(baseline_shots, baseline_precision, 'b-', label='Baseline')
74 ax1.plot(semantic_shots, semantic_precision, 'g-', label='Semantic Similarity')
75 ax1.plot(prompt_opt_shots, prompt_opt_precision, 'r-', label='Prompt
    Optimization')
76 ax1.scatter(baseline_shots, baseline_precision, marker='o', color='b')
77 ax1.scatter(semantic_shots, semantic_precision, marker='^', color='g')
78 ax1.scatter(prompt_opt_shots, prompt_opt_precision, marker='s', color='r')
79 ax1.set_xlabel('Number of Shots')
80 ax1.set_ylabel('Precision')

```



```

81 ax1.set_title('Precision vs. Number of Shots')
82 ax1.legend()
83 ax1.grid(True)
84
85 # Recall
86 ax2.plot(baseline_shots, baseline_recall, 'b-', label='Baseline')
87 ax2.plot(semantic_shots, semantic_recall, 'g-', label='Semantic Similarity')
88 ax2.plot(prompt_opt_shots, prompt_opt_recall, 'r-', label='Prompt Optimization')
89 ax2.scatter(baseline_shots, baseline_recall, marker='o', color='b')
90 ax2.scatter(semantic_shots, semantic_recall, marker='^', color='g')
91 ax2.scatter(prompt_opt_shots, prompt_opt_recall, marker='s', color='r')
92 ax2.set_xlabel('Number of Shots')
93 ax2.set_ylabel('Recall')
94 ax2.set_title('Recall vs. Number of Shots')
95 ax2.legend()
96 ax2.grid(True)
97
98 # F1 Score
99 ax3.plot(baseline_shots, baseline_f1, 'b-', label='Baseline')
100 ax3.plot(semantic_shots, semantic_f1, 'g-', label='Semantic Similarity')
101 ax3.plot(prompt_opt_shots, prompt_opt_f1, 'r-', label='Prompt Optimization')
102 ax3.scatter(baseline_shots, baseline_f1, marker='o', color='b')
103 ax3.scatter(semantic_shots, semantic_f1, marker='^', color='g')
104 ax3.scatter(prompt_opt_shots, prompt_opt_f1, marker='s', color='r')
105 ax3.set_xlabel('Number of Shots')
106 ax3.set_ylabel('F1 Score')
107 ax3.set_title('F1 Score vs. Number of Shots')
108 ax3.legend()
109 ax3.grid(True)
110
111 plt.tight_layout()
112 plt.savefig('f1_vs_shots_prompt_opt_combined.png')
113 plt.close()

```

References

- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. URL <https://arxiv.org/abs/2005.14165>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.
- Pengfei Liu, Weijie Peng, Yuning Mao, Liyang Li, Xiangkun Liu, Tianxiang Gao, Xu Han, Zhengyan Zhang, Ning Ding, Huayang Li, Zhengdong Lu, Zhiyuan Liu, and Jie Tang. Pre-train, prompt,

and predict: A systematic survey of prompting methods in natural language processing. *ACM Comput. Surv.*, 55(9):1–135, 2023. doi: 10.1145/3560829. URL <https://doi.org/10.1145/3560829>.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019. URL <https://arxiv.org/abs/1907.11692>.

Archit Parnami and Minwoo Lee. Learning from few examples: A summary of approaches to few-shot learning, 2022. URL <https://arxiv.org/abs/2203.04291>.